KAFKA, MADE SIMPLE

Apache Kafka

A simple guide to events, pub/sub, clusters, and handling failures

Includes a small Spring Boot demonstration

Beginner-friendly training session

START HERE

Today's learning path

1. Why Kafka?

- The problem it solves
- Benefits and trade-offs
- Simple real-life example

2. How it works

- Topics, partitions, brokers
- Producers, consumers, groups
- Offsets and replication

3. Use it well

- Event-driven design
- Single broker vs cluster
- Retries, DLQ, and advanced tools

4. See it work

- Spring Boot publisher
- Two subscriber services
- Failure sent to a DLQ

Goal: explain the ideas in plain language, then see them in a running example.

START WITH AN EXAMPLE

An event is a fact that already happened

An everyday example

- A customer placed an order.
- That fact is an event: OrderCreated.
- Email, stock, and reports can react later.

A useful event has

- A unique event or order ID.
- A business key, such as orderId.
- A timestamp and schema version.
- Only the data consumers need.

Event vs command

- Event: OrderCreated tells us what happened.
- Command: ReserveStock asks for an action.
- Kafka can carry both; agree on meaning.

THE PROBLEM

Why do we need Kafka?

Many systems need the same fact

- An order arrives once.
- Inventory, billing, and email all need to react.
- Direct calls make the order service know every destination.

Systems are not always ready

- A receiving service may be slow or offline.
- A direct call can fail the whole request.
- A queue can hold the work until it is read.

We may need the history

- A new service may need old events.
- A bug fix may require replaying events.
- Kafka keeps records for a configured time or size.

Kafka is useful when many services need reliable, replayable streams of data.

THE SIMPLE PICTURE

What is Kafka?

Think of a shared event journal

- A producer writes a record.
- Kafka stores records in a topic.
- Consumers read records when they are ready.

Kafka is a distributed system

- Data is split across partitions.
- Partitions can be copied to other brokers.
- A group of brokers forms a Kafka cluster.

Not just a queue

- Records can remain after they are read.
- Different consumer groups can read independently.
- Consumers can often rewind and read again.

A topic is like a named stream of related events.

What are the benefits of Kafka?

Scale

- Partitions let work run in parallel.
- Add consumers in a group to share work.
- Add brokers to spread storage and traffic.

Resilience

- Replication can protect data from a broker failure.
- Consumers can catch up after being offline.
- Producer and consumer failures can be handled separately.

Reuse and replay

- One event can serve many independent teams.
- New consumers can start from retained history.
- Useful for analytics and integrations.

Remember the cost

- Kafka needs operations and monitoring.
- It adds eventual, not instant, processing.
- Do not use it just to make a tiny app fashionable.

When should we use Kafka?

A good fit

- Many services need the same events.
- Traffic arrives in bursts.
- Readers need replay or independent progress.
- Analytics and real-time pipelines.

A simpler option may fit

- Need an immediate answer? Consider HTTP.
- A tiny background task may only need a queue.
- Kafka adds storage, monitoring, and operating work.

Words you will hear

People and places

- Producer: writes records.
- Consumer: reads records.
- Broker: one Kafka server.
- Cluster: brokers working together.

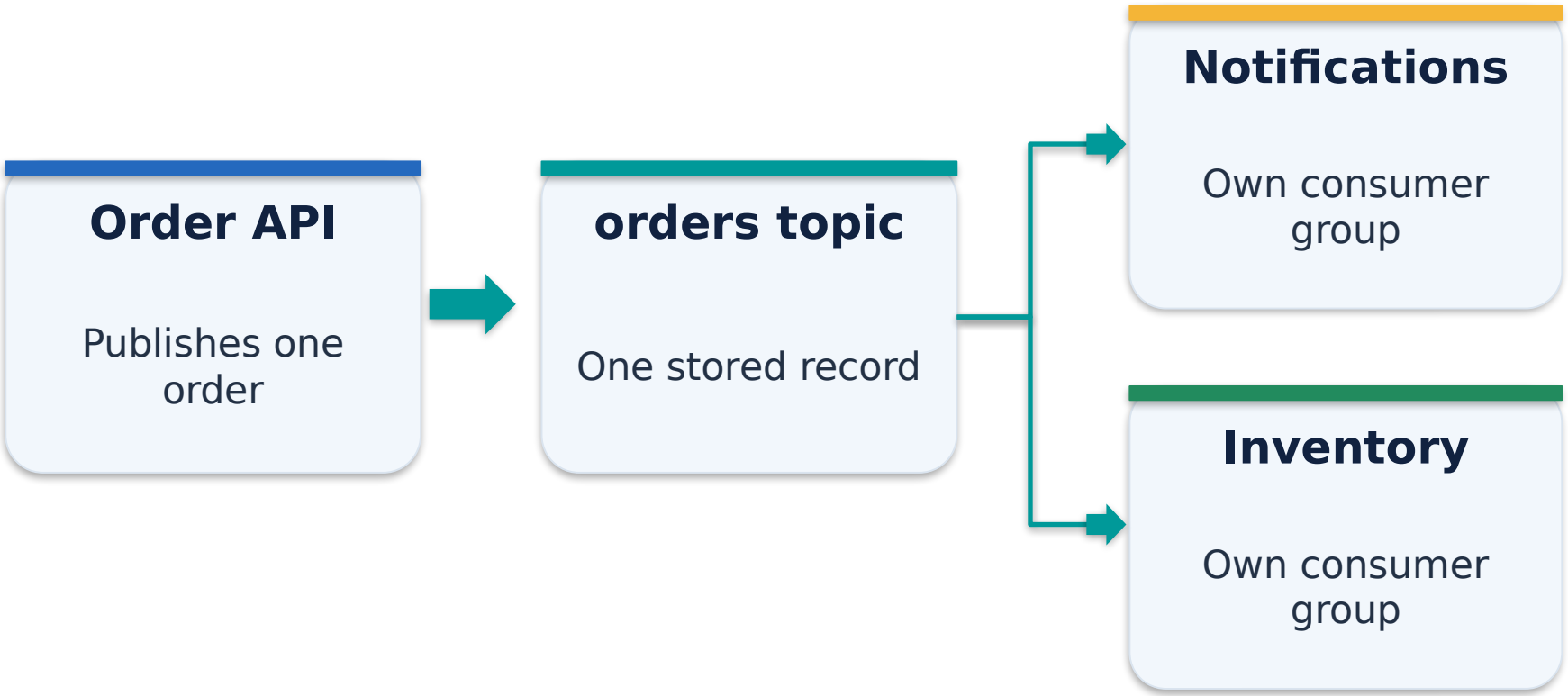
Data organization

- Topic: named stream, like orders.
- Partition: one ordered piece of a topic.
- Record: key, value, time, and headers.
- Offset: record position in a partition.

Reading together

- Consumer group: consumers sharing work.
- Each group tracks its own offsets.
- Different groups can each see the full stream.

Pub/sub: publish once, many can listen



Both groups read the same event.

Inventory publishes a new event. Shipping listens to it.

One published event can be read independently by multiple services.

Consumer groups: copies or shared work?

Different group IDs

- Each group gets its own view of the topic.
- Email and inventory both see each order.
- Each group commits its own offsets.

Same group ID

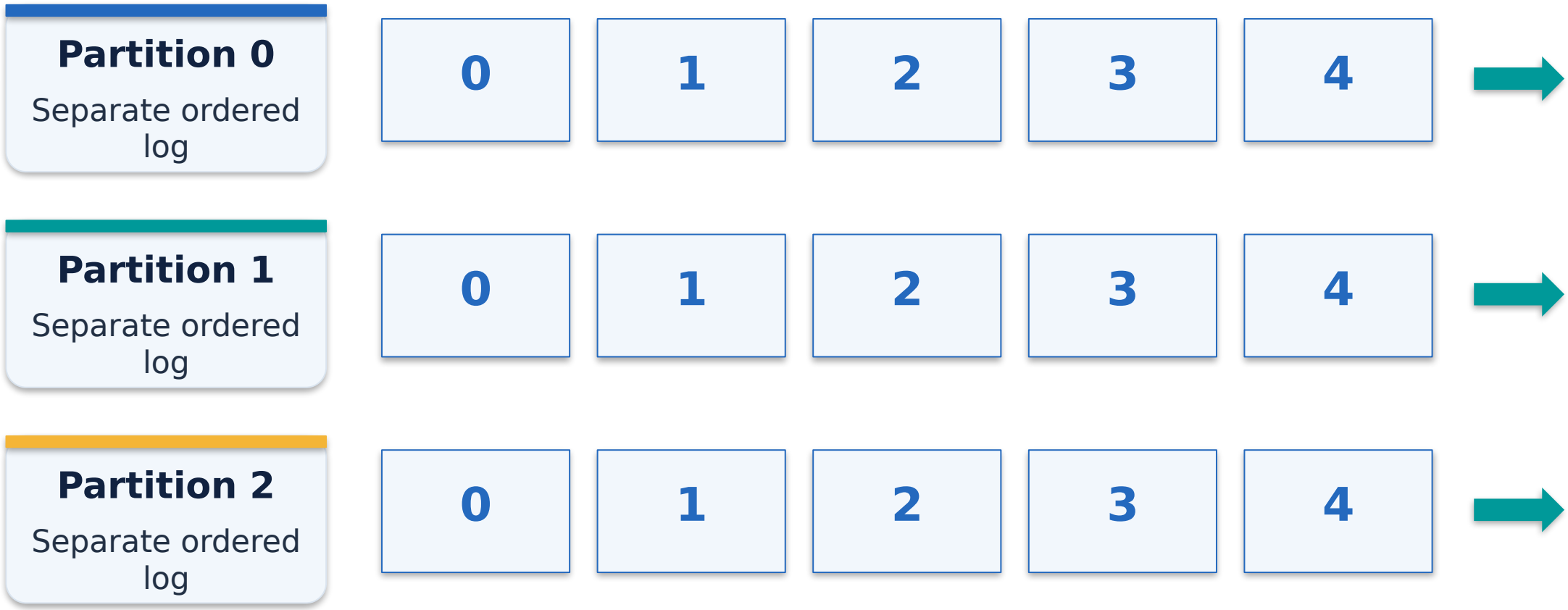
- Members share the partitions.
- One partition is assigned to one member at a time.
- Each record is handled by one member in that group.

Important limit

- For standard consumer groups: active readers are limited by partitions.
- Extra consumers may sit idle.
- More partitions can allow more parallel work.

Different groups mean fan-out; members in one group mean load sharing.

How Kafka stores data inside



Numbers are offsets. New records append on the right.

What happens when a producer sends?

1. Build a record

- Choose a topic.
- Set a value and optional key.
- Add headers if useful.

2. Pick a partition

- A key usually sends related records to the same partition.
- Without a key, the producer can spread records.
- The partition determines the ordering lane.

3. Send and confirm

- Batching and compression can improve efficiency.
- Kafka acknowledges according to the chosen setting.
- The producer can retry temporary errors.

Keys help keep related records together; acknowledgement settings affect safety and speed.

How records reach disk efficiently

Producer

- Buffers records and sends batches.
- Compression makes batches smaller.
- Metadata tells the client which broker leads.

Broker

- Appends batches to segment files.
- Uses the operating system page cache.
- Indexes help locate offsets and times.

Consumer

- Pulls batches with poll().
- Reads from partition leaders by default.
- Reading does not remove a record.

Sequential logs, batching, and caching help Kafka handle large streams efficiently.

What happens when a consumer reads?

Read

- A consumer joins a group.
- The group coordinates partition assignments.
- The consumer polls records.

Process

- The application handles each record.
- It may call a database or another service.
- Slow work can delay later records in that partition.

Save progress

- The group stores an offset.
- On restart, it continues from its committed position.
- Commit only after the work you need is complete.

An offset is a bookmark; the application controls when progress is saved.

Offsets: a bookmark for each partition

A record position

- Partition 0: offsets 0, 1, 2, 3, 4.
- Partition 1 has its own 0, 1, 2…
- Offset 4 in P0 is unrelated to offset 4 in P1.

Committed offset

- Processed P0 offsets 0 to 4? Commit 5.
- 5 is the next record the group expects to read.
- Each group has separate bookmarks.

After a restart

- Resume from the committed offset.
- Uncommitted work can repeat.
- Kafka stores group bookmarks in __consumer_offsets.

Committed offset means the next position to read; offsets are not global message IDs.

IMPORTANT DESIGN CHOICE

Partitions: parallelism and ordering

Ordering

- Kafka preserves order inside one partition.
- It does not promise one total order across a topic.
- Use the same key for records that must stay together.

Example key

- Key = customerId.
- All events for one customer go to one partition.
- Different customers can be processed in parallel.

Choose carefully

- Partition count affects parallelism.
- Too few can limit throughput.
- Adding partitions later can change key-to-partition mapping.
- Membership changes can trigger a group rebalance.

Decide what must stay in order before selecting the key and partition count.

Lag: how far is a reader behind?

Simple example

- Log end offset = 100.
- Group committed offset = 90.
- Committed lag = 100 - 90 = 10 positions.

Why it grows

- Consumers are slow or stopped.
- A bad record keeps failing.
- Traffic grew, or one partition is overloaded.

How to respond

- Check processing time and errors.
- Add consumers up to the partition limit.
- Watch event age as well as lag count.

Lag is measured per group and partition; it does not by itself prove data loss.

Rebalancing and multiple app instances

Three partitions, one app

- One member owns P0, P1, and P2.
- It can process all records.
- Partitions stay separate ordering lanes.

Three partitions, two apps

- Same group ID: partitions are shared.
- One app may get two; the other gets one.
- A fourth member would be idle with three partitions.

When members change

- Joining or leaving can rebalance ownership.
- Processing may pause during reassignment.
- Do not exceed max.poll.interval.ms with slow work.

More application instances share work; they do not create more broker replicas.

Kafka with one broker

Good for learning

- One Kafka server runs on one machine.
- Simple to start with Docker or locally.
- A Spring Boot demo can use one broker.

What still works

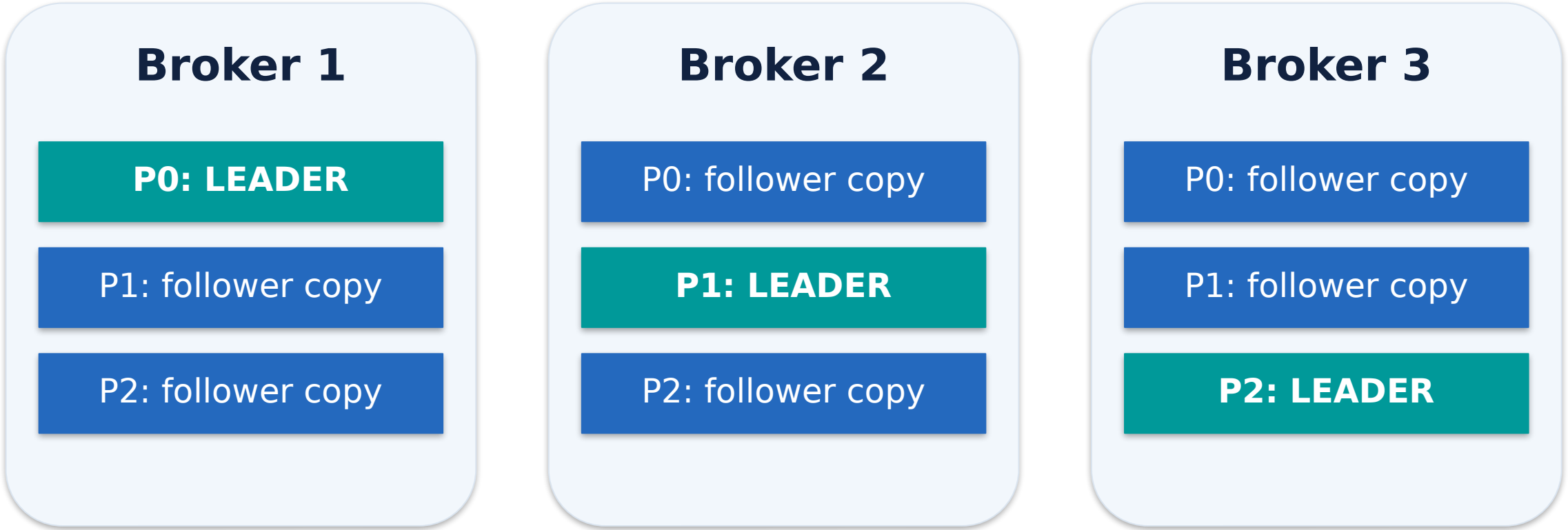
- Topics, partitions, producers, consumers.
- Consumer groups and offsets.
- Retries and a DLQ can be demonstrated.

What it cannot do

- No broker redundancy.
- A broker or machine failure stops service.
- Replication factor 1 is not fault tolerant.

A single broker is a teaching setup, not high availability.

Kafka with multiple brokers



One leader per partition. Three copies per partition. Leaders can be spread across brokers.

One broker vs three brokers

One local broker

- Replication factor = 1.
- Easy to learn and run.
- If it stops, there is no other data replica.

Three-broker lab

- Replication factor = 3.
- Minimum in-sync replicas = 2.
- A healthy remaining pair can keep writing.

Real deployment

- Spread replicas across hosts or zones.
- Use a controller quorum, commonly 3 or 5.
- Combined broker/controllers here are a lab shortcut.

Three containers on one laptop demonstrate replication, not host-level disaster protection.

ZooKeeper and KRaft: old and new

Older installations

- ZooKeeper was a separate coordination system.
- It helped manage metadata and controllers.
- You may still find it in older tutorials.

Modern Kafka

- KRaft means Kafka uses Raft for metadata.
- Controllers keep a replicated metadata log.
- Kafka 4.0 removed ZooKeeper support.

Controller vs partition leader

- Controllers manage cluster metadata.
- Partition leaders handle reads and writes for data.
- They are different responsibilities.

The included Kafka 4.0.0 labs use KRaft, with no ZooKeeper container.

Replication, acks, and durability

Replication factor

- Number of stored copies for a partition.
- Often 3 in production, if the cluster supports it.
- Copies should be on different brokers.

Producer acks

- acks=0: do not wait for broker confirmation.
- acks=1: wait for the leader.
- acks=all: wait for all current in-sync replicas.

ISR and min ISR

- ISR means replicas caught up enough to count.
- min.insync.replicas can reject risky writes.
- The right settings balance availability and safety.

Replication helps only when placement and acknowledgement settings are sensible.

Leader failure: what happens next?

Before a failure

- P0 leader is on broker 1.
- Brokers 2 and 3 hold follower replicas.
- Followers fetch and copy the leader log.

Broker 1 stops

- The controller selects an eligible in-sync replica.
- Clients refresh metadata and retry.
- A brief delay is normal during recovery.

Safety settings matter

- RF=3, min ISR=2, acks=all tolerates one broker loss.
- With only one ISR left, writes are rejected.
- A metadata quorum must also remain available.

Failover depends on healthy replicas, a controller majority, and correct configuration.

Producer safety and speed settings

Safety

- acks=all plus min ISR protects acknowledged writes.
- enable.idempotence avoids retry duplicates in Kafka.
- Check send results; do not ignore async errors.

Speed

- batch.size controls batch size limits.
- linger.ms allows time to build batches.
- compression.type reduces network and disk use.

Limits

- delivery.timeout.ms bounds delivery waiting.
- Idempotent producer does not deduplicate business orders.
- Measure changes with realistic traffic.

Tune safety first, then measure latency and throughput.

Retention and compaction

Time/size retention

- Keep records for a configured time or space.
- Old data expires whether read or unread.
- Choose retention for replay and storage needs.

Log compaction

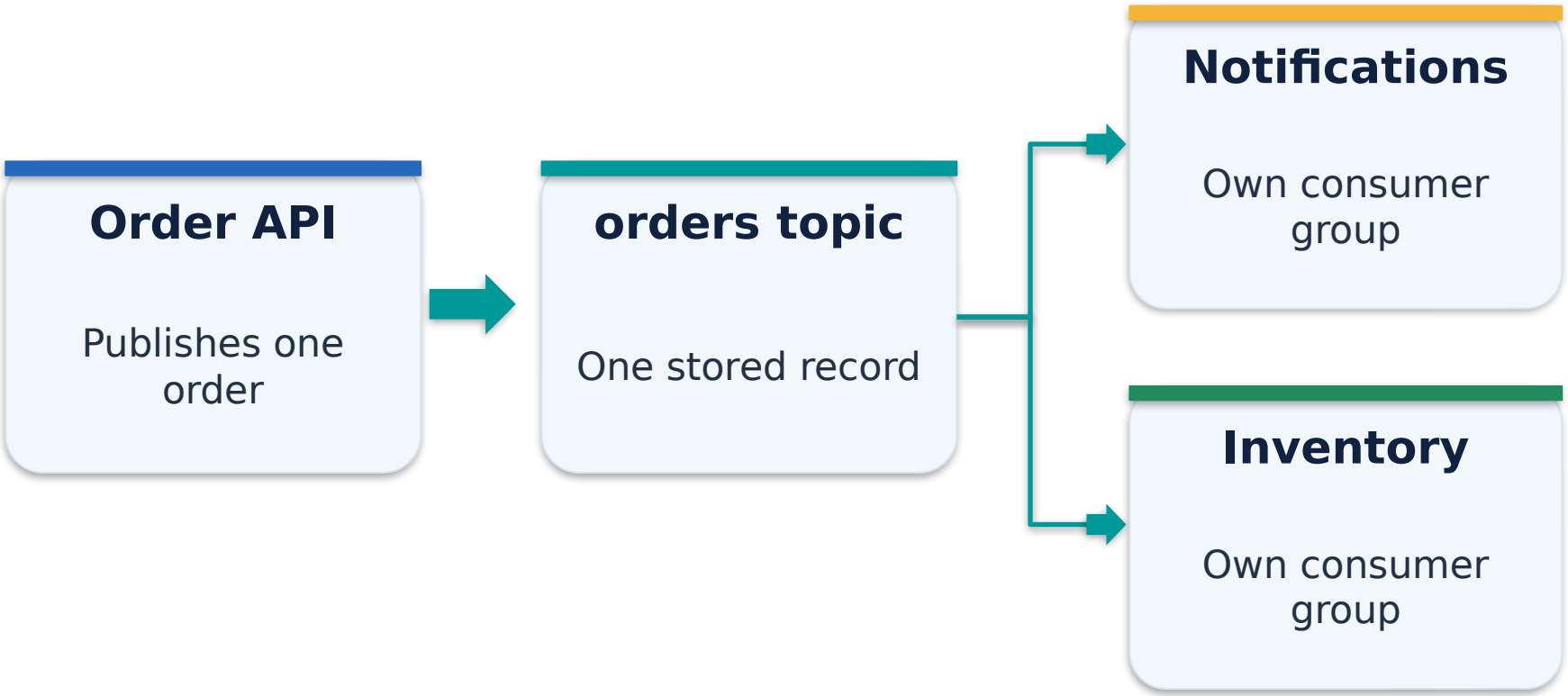
- Keep the latest value for each key over time.
- Useful for the latest state of an entity.
- It is not the same as a normal database table.

Plan storage

- Estimate event size and incoming rate.
- Include replicas in disk estimates.
- Watch disk usage and consumer lag.

Kafka is durable storage for a configured window, not infinite storage by default.

Event-driven architecture with Kafka



Both groups read the same event.

Inventory publishes a new event. Shipping listens to it.

Services react to facts that happened, instead of calling every other service.

Event-driven: good habits

Name events as facts

- OrderCreated, PaymentReceived.
- Use past-tense names where practical.
- Do not expose internal database objects as events.

Expect repeats

- A record may be delivered again after a crash.
- Make important processing idempotent.
- Use an event ID or business key to detect duplicates.

Plan for change

- Version event schemas carefully.
- Keep consumers tolerant of compatible changes.
- Track event ownership and meaning.

A message bus does not remove the need for careful service design.

Event-driven workflows and eventual consistency

A chain of facts

- OrderCreated -> StockReserved -> ShipmentPrepared.
- Each step publishes its result.
- Other services react when results arrive.

Expect time gaps

- HTTP 202 means accepted, not business completion.
- Show pending status while work continues.
- Use event IDs to trace a full journey.

When a step fails

- A saga can coordinate several business steps.
- A compensation may release reserved stock.
- A technical retry cannot fix every business failure.

Events and the database: avoid lost updates

The dual-write problem

- Save an order, then publish an event.
- A crash between the two can lose the event.
- Publishing first can announce an unsaved order.

Transactional outbox

- Save the order and an outbox row together.
- A relay or CDC connector publishes outbox rows.
- Use event IDs to make repeated delivery safe.

Demo boundary

- This demo has no database or real side effects.
- One app contains several logical services.
- Production services can deploy independently.

An outbox bridges a database transaction and event publishing; Kafka alone cannot do both atomically.

Delivery guarantees in plain words

At-most-once

- A record is handled zero or one time.
- Can lose work if progress is saved too early.
- Useful only when occasional loss is acceptable.

At-least-once

- Retry until work succeeds.
- A record can be seen more than once.
- Common choice; make processing idempotent.

Exactly-once

- Kafka transactions can make Kafka-to-Kafka steps atomic.
- This does not magically make an external database/API exactly once.
- External side effects still need a safe design.

Be precise: 'exactly once' has a scope and is not a magic setting.

Transactions and exactly-once: know the scope

Kafka transaction

- Can publish records and consumed offsets atomically.
- Consumers use read_committed to hide aborted records.
- Useful for Kafka-to-Kafka workflows.

Outside Kafka

- An email or payment API is a separate side effect.
- Kafka transactions do not make it exactly once.
- Use idempotency keys and a safe storage design.

This lab

- Uses ordinary at-least-once processing.
- A crash can repeat shipping or DLQ records.
- Logs simulate work; there is no payment or email API.

Exactly-once guarantees apply to a defined boundary, not to every connected system.

DLQ: what it is and what it is not

DLQ means dead-letter queue

- In Kafka, it is commonly another topic.
- Example: orders.DLT.
- It holds records that could not be processed normally.

Kafka does not do it automatically

- Your consumer/framework detects the failure.
- It can retry, then publish to the DLQ.
- Spring Kafka provides a recoverer for this pattern.

A DLQ is not a trash can

- Keep the original event and failure details.
- Alert and inspect the DLQ.
- Fix, replay, or safely discard with a reason.

A DLQ makes failures visible; it does not fix them by itself.

Retry choices: temporary or permanent failure?

Temporary failure

- A network or service may recover soon.
- Retry a limited number of times with backoff.
- The lab uses 1 attempt plus 2 retries.

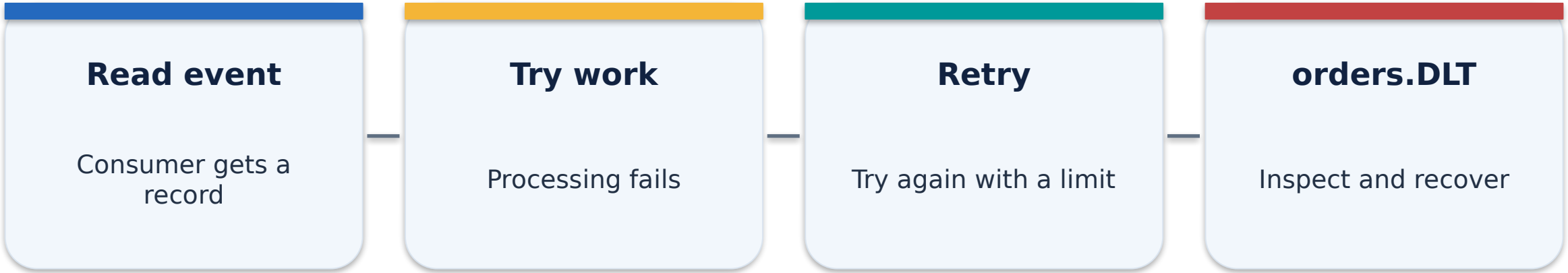
Permanent failure

- Bad data usually fails again unchanged.
- Route it for inspection without endless retry.
- Amount 0 is retried only to make the lab visible.

Two retry styles

- Blocking retries delay work in that partition.
- Retry topics let other work move forward.
- Retry topics can change processing order.

A simple retry-to-DLQ flow



In the demo: one first attempt plus two retries, then the record goes to orders.DLT.

DLQ operations: inspect, repair, replay

Keep enough context

- Original payload and key.
- Source topic, partition, and offset.
- Exception details and a correlation ID.

Recover safely

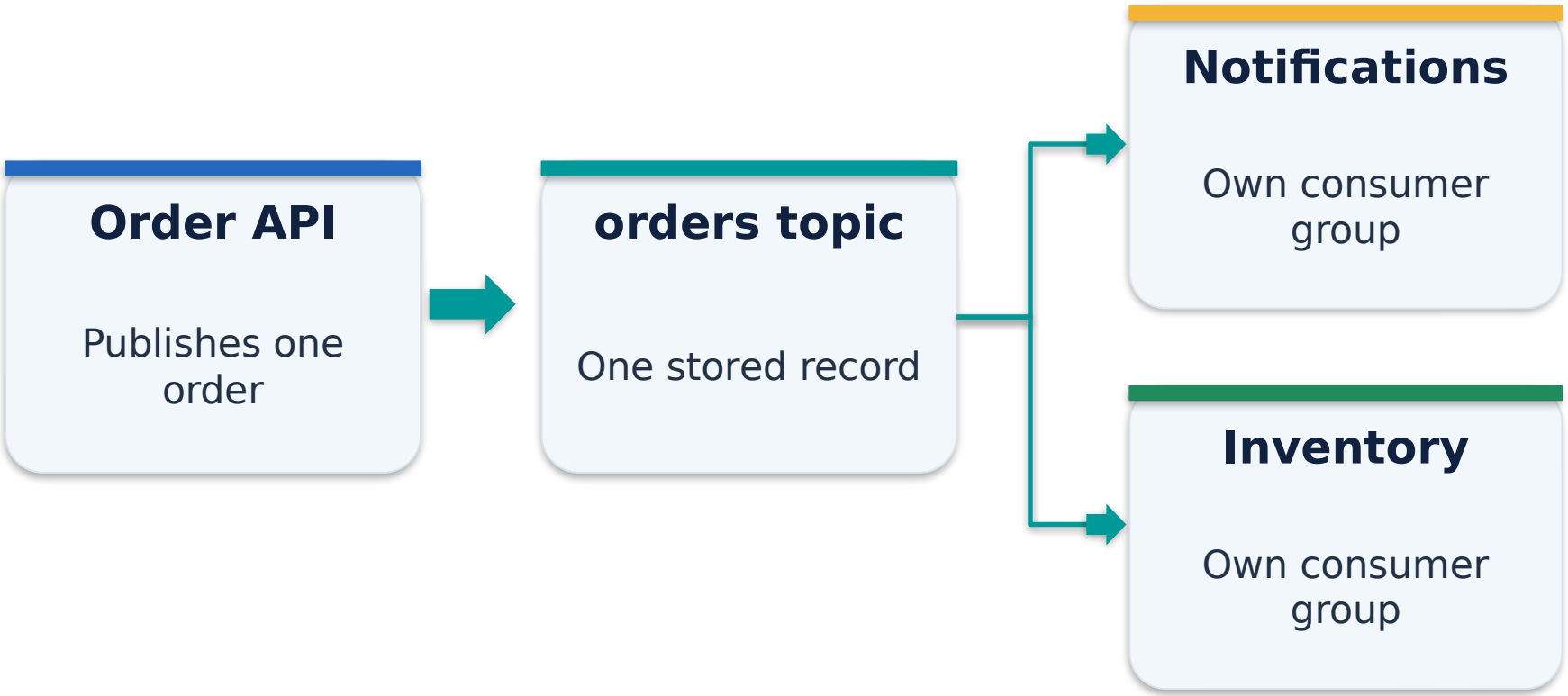
- Alert someone and find the cause.
- Fix code or validate a corrected event.
- Replay carefully with duplicate protection.

Avoid new problems

- Do not replay blindly into the same failure.
- Set DLQ retention and access rules.
- If DLQ publishing fails, do not skip the source record.

The demo logs DLQ headers; reading a DLQ does not delete or repair its records.

The Spring Boot demo



Both groups read the same event.

Inventory publishes a new event. Shipping listens to it.

Two consumer groups each see an event; only the inventory handler rejects amount 0.

LIVE DEMONSTRATION

Run the demo

Start Kafka

- Run: docker compose up -d --wait
- One local broker starts.
- Wait for the broker before starting the app.

Start Spring Boot

- In demo/: mvn spring-boot:run
- POST an order with amount 25.50.
- See notification and inventory logs.

Trigger the DLQ

- POST another order with amount 0.
- Inventory retries, then fails.
- See the orders.DLT log.

Copy-paste commands and the full demo explanation are in README.md.

LIVE DEMONSTRATION

Run the three-broker and two-app labs

Three brokers

- Use compose.cluster.yaml.
- Bootstrap ports: 29092, 29093, 29094.
- Start app with replication factor 3.

Two Spring Boot instances

- Use ports 8080 and 8081.
- Keep the same listener group IDs.
- Three partitions are shared across the two apps.

Verify

- Describe partitions, leaders, and ISR.
- Inspect the consumer groups and lag.
- Stop a broker, publish again, then restart it.

Exact commands and expected observations are in docs/DEMO-GUIDE.md.

Advanced Kafka features

Kafka Connect

- Moves data between Kafka and other systems.
- Source connectors bring data in.
- Sink connectors send data out.

Kafka Streams

- Java library for processing event streams.
- Filter, transform, join, and aggregate.
- Can keep local state and recover it.

Producer features

- Idempotence helps prevent duplicate writes from producer retries.
- Transactions can make Kafka-to-Kafka writes atomic.
- Batching and compression can improve efficiency.

Platform tools

- Schema Registry is a commonly used separate service.
- MirrorMaker 2 can copy data between clusters.
- Modern Kafka uses KRaft for cluster metadata.

Some tools are built into Kafka; Schema Registry is commonly deployed separately.

Kafka Connect and change data capture

Connect

- A separate runtime for source and sink connectors.
- A source writes to Kafka; a sink reads from Kafka.
- Distributed workers share connector tasks.

CDC

- Change data capture watches database changes.
- A connector such as Debezium can publish them.
- Often used to relay an outbox reliably.

Plan the details

- Connectors must be installed and configured.
- Compatibility and delivery guarantees vary.
- Monitor task failures and connector lag.

Kafka Connect is the integration framework; a connector supplies system-specific behavior.

Kafka Streams, windows, and state

Simple processing

- Filter invalid events.
- Transform an order into another event.
- Group and count orders per customer.

Time-based work

- A window groups events over a time range.
- Event time can differ from arrival time.
- Grace periods allow some late events.

State and joins

- State stores keep running totals or lookup data.
- Changelog topics help rebuild state.
- Joins may require matching partitioning.

Kafka Streams is a Java library in your application, not a separate Kafka broker service.

Schemas and event evolution

A contract

- A schema describes the event fields.
- JSON, Avro, and Protobuf are common choices.
- Document field meaning, not only field types.

Change safely

- Prefer compatible additions with defaults.
- Do not casually rename or remove fields.
- Test old readers with new records and vice versa.

Schema Registry

- A commonly added separate service.
- Stores schemas and checks compatibility rules.
- It is not built into the Apache Kafka broker.

The lab uses simple JSON; a registry is an optional ecosystem component.

Security and data protection

Protect the connection

- TLS encrypts traffic.
- SASL or certificates identify clients.
- ACLs control who can read or write topics.

Protect the data

- Keep secrets out of event payloads.
- Plan retention, deletion, and access reviews.
- Limit who can read failure details in a DLQ.

Lab vs production

- Local labs use plaintext for simplicity.
- Use private networking and secure listeners at work.
- Do not expose these lab ports publicly.

Monitoring and performance

Watch consumers

- Lag and event age.
- Processing time, retries, and DLQ rate.
- Frequent rebalances or stalled polling.

Watch brokers

- Disk space and storage growth.
- Under-replicated or offline partitions.
- Request latency and controller quorum health.

Improve carefully

- Check skew: one hot key can overload a partition.
- Estimate storage: bytes/sec x retention x replicas.
- Leave space for overhead and recovery.

More partitions cannot split one hot key while preserving that key's strict order.

Replay and disaster recovery

Replay retained records

- Use a new group to read retained history.
- Or stop a group and reset its offsets carefully.
- earliest is used only when no valid offset exists.

Limits

- Expired records cannot be replayed from Kafka.
- Compacted logs may not contain every old value.
- Replay can repeat real business actions.

Another cluster

- MirrorMaker 2 can replicate between clusters.
- Replication is usually asynchronous.
- Plan failover, offsets, and possible gaps or repeats.

Replication within a cluster is not a complete backup or cross-region recovery plan.

DISCUSSION

Quick knowledge check

Ask the audience

- Two different groups: who sees the order?
- Four consumers and three partitions: who works?
- Does reading delete a Kafka record?

More questions

- If committed offset is 8, what reads next?
- Does Kafka route failures to a DLQ by itself?
- Does one broker give high availability?

Answers

- Both groups; at most three active consumers.
- No deletion; next offset is 8 if retained.
- No automatic DLQ; no single-broker failover.

Use these questions before the recap to check understanding.

TAKE IT TO WORK

Production checklist and recap

Design

- Pick meaningful event names and keys.
- Set partitions, retention, and schema rules.
- Make consumers idempotent and safe to replay.

Operate

- Use multiple brokers and suitable replication.
- Secure with TLS, authentication, and ACLs.
- Monitor lag, disk, under-replicated partitions, and DLQ.

Remember

- Kafka stores and distributes event streams.
- Groups decide fan-out versus shared work.
- DLQ handling is configured by your application.